PROJECT PRODUCTION DOCUMENTATION: SENTIMENTOPS

Distributed Sentiment Analytics Engine — Systems Learning Ledger (v2: Post-Hardening Revision)

System Domain:	Distributed Sentiment Analytics Pipeline
Tech Stack:	FastAPI, Celery, Upstash Redis, Neon PostgreSQL, Docker
Current Deployment:	Render (Web Service) + On-Demand Local Worker/Beat
Original Deployment:	Railway Cloud (3 continuous containers)
Data Volume:	19,000+ production reviews

Revision Note: This document supersedes the original SentimentOps production log. It preserves the original five deployment bugs found while running continuously on Railway, and adds two further engineering fixes — a security hardening pass and a cache-invalidation redesign — made after a deliberate move to free-tier hosting. The Current Deployment Model section below states plainly what runs continuously today versus what runs on demand. Nothing in this document overstates uptime that doesn't exist.

1. Current Deployment Model

The system is no longer a permanently-running 3-service cluster. The table below states exactly what is live at all times versus what is run on demand, and why.

Component	Where It Runs	Behavior
FastAPI (Web)	Render — Free Web Service	Always reachable. Kept warm by a scheduled GitHub Actions ping to /health every 10 min.
Celery Worker	Local machine, on-demand	Render's free tier has no Background Workers; Railway's free worker hours expired. Started manually to refresh data or recompute cache.
Celery Beat	Not continuously running	Same constraint as the worker. Scheduled recompute is triggered manually, not by a live 24/7 cron daemon.
Upstash Redis	Always live (managed)	Holds the last cached analytics payload indefinitely until manually refreshed.
Neon PostgreSQL	Always live (managed)	Holds all 19,000+ rows permanently, independent of any compute host's uptime.
Frontend (Lovable)	Always live	Displays whatever is cached in Redis — a snapshot of the last computed run, not a continuously live-updating feed.

Why this matters: the API and dashboard are reachable at any time and show real, previously computed data. The background compute layer (worker + Beat) is not always running, because Render's free tier has no Background Worker option and Railway's free worker allowance ran out. Running all three services continuously is a small monthly cost that has not yet been incurred for this portfolio-stage project. The system can be brought fully live on demand by starting the worker locally against the same Upstash Redis and Neon Postgres instances the deployed API already uses.

2. System Architecture (Full 3-Service Design)

This is the architecture as originally deployed and validated, continuously, on Railway. The component responsibilities below remain accurate; only the current hosting continuity has changed, as described in Section 1.

```
[Client / React Dashboard]
        |
        v
Sentiment-Web (FastAPI)
   serves REST, issues 202 Accepted immediately
        |
        v
Upstash Redis
   dual role: Celery broker + analytics cache
   |                          |
   v                          v
Sentiment-Worker          Sentiment-Beat
(Celery, -P solo)         (Celery Beat scheduler)
   |                          |
   v                          v
Neon PostgreSQL            Broadcasts:
(19,000+ rows)              compute_global_stats
                             compute_distribution
                             compute_urgent_reviews
```

3. The Production Engineering Ledger

Bugs 1 through 5 were identified during the original continuous Railway deployment. Bugs 6 and 7 were identified and fixed afterward, during a security and architecture hardening pass.

BUG 1: ASYNCHRONOUS EVENT-LOOP STARTUP DEADLOCK

Symptom: FastAPI froze on deployment, hanging indefinitely at the lifecycle line “Waiting for application startup” before being terminated by the platform's health check timeout.

Root Cause: The startup hook (`@app.on_event("startup")`) ran a schema verification call, `await conn.run_sync(Base.metadata.create_all)`, on every container boot. This DDL-style operation interacted poorly with connection setup during startup, delaying socket binding past the health check window.

Solution: Schema creation was moved to a dedicated one-time migration script (`force_neon_init.py`), run manually rather than on every container start. The startup hook now performs no database call at all.

BUG 2: SERVERLESS CONNECTION POOL STALLS VIA PROXY GATEWAYS

Symptom: Database query channels routinely locked up mid-session during data processing without triggering standard diagnostic database exceptions.

Root Cause: Neon routes connections through a PgBouncer proxy in transaction pooling mode. The asyncpg driver generates server-side prepared statements by default. In transaction pooling, sequential operations within a session are distributed dynamically across different backend clusters — referencing a prepared statement on an instance that didn't generate it caused a silent deadlock.

Solution: Disabled the driver's prepared statement cache entirely in database.py:

```
engine = create_async_engine(
DATABASE_URL,
prepared_statement_name_cache_size=0
)
```

BUG 3: CROSS-OS RUNTIME CACHE CORRUPTION (WinError 3)

Symptom: The background execution daemon crashed continuously with system trace warnings pointing to “WinError 3: The system cannot find the path specified.”

Root Cause: Celery Beat generates local state tracker binaries (celerybeat-schedule.dat, .bak) during development. On a Windows machine, these binaries embedded absolute Windows disk paths. Pushing them to source control caused the Linux cloud container to read the Windows paths and throw fatal exceptions.

Solution: Removed the binaries from the Git index and added explicit pattern exclusions to .gitignore:

```
sentimentops-backend/celerybeat-schedule*
pipeline.log
__pycache__/
```

BUG 4: CONTAINER WORKSPACE NAMESPACE COLLISIONS (Missing Modules)

Symptom: The web container booted correctly; worker instances collapsed instantly with “ModuleNotFoundError: No module named 'database'.”

Root Cause: Application logic lives inside a subfolder (sentimentops-backend/). The Dockerfile set WORKDIR /app. When Celery spawned internal computation threads, it resolved module lookups from /app, losing visibility of sibling modules like database.py inside the subfolder.

Solution: Set WORKDIR directly to the subfolder and bound PYTHONPATH to the same path inside the Dockerfile, so the fix survives regardless of which process (Uvicorn, Celery worker, Beat) starts the container:

```
ENV PYTHONPATH=/app/sentimentops-backend
WORKDIR /app/sentimentops-backend
CMD ["sh", "-c", "uvicorn app:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

BUG 5: BROWSER PREFLIGHT ACCESS RESTRICTIONS (CORS Stalls)

Symptom: Cloud container logs reported clean data operations, but the interactive frontend remained entirely blank, with a browser console exception: “Response to preflight request doesn't pass access control check.”

Root Cause: Lovable generates dynamic sandbox subdomains (*.lovableproject.com) for frontend previews. The backend's strict CORS rules rejected the dynamic domain, blocking client-side data parsing.

Solution: Adjusted CORS middleware to allow all origins — acceptable here since the backend serves only aggregate, read-only analytics with no sensitive or write-privileged data exposed:

```
app.add_middleware(
CORSMiddleware,
allow_origins=["*"],
allow_credentials=False,
allow_methods=["*"],
allow_headers=["*"],
)
```

BUG 6: HARDCODED CREDENTIALS IN SOURCE CODE (New)

Symptom: Database and Redis connection strings, including live passwords, were committed directly into database.py, celery_app.py, and force_neon_init.py as default fallback values inside os.getenv("VAR", "real-credential") calls — exposing them in the public GitHub repository.

Root Cause: Using a literal connection string as the fallback argument to os.getenv() means that value is used whenever the environment variable is unset, and remains permanently visible in source control regardless of whether production configuration is correct.

Solution: Removed all hardcoded fallback values. Every credential now fails loudly at startup if missing, instead of silently falling back to an exposed value:

```
DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
raise RuntimeError("DATABASE_URL environment variable not set")
```

All exposed credentials (Upstash Redis password, Neon Postgres password) were rotated immediately after removal from source.

BUG 7: CACHE INVALIDATION STORM UNDER BULK INGESTION (New)

Symptom: Upstash Redis usage was exhausted well before any heavy traffic justified it.

Root Cause: The single-review ingestion endpoint called redis_client.delete() on the stats cache key on every insert. During bulk ingestion of 19,000+ rows at roughly 80 inserts/sec, this invalidated the cache about 80 times per second; every subsequent read became a cache miss, each separately enqueueing a new Celery recompute task and compounding load far beyond actual read traffic.

Solution: Removed per-row cache invalidation from the single-review endpoint. Recomputation is now triggered exactly once, after a full batch finishes inserting, rather than per row or on a fixed timer:

```
if db_insert_list:
await session.execute(insert(Sentiment), db_insert_list)
await session.commit()
compute_global_stats.delay()
compute_distribution.delay()
compute_urgent_reviews.delay()
```

A Redis-backed lock (SET key val NX EX 10) was added to each analytics read route to prevent duplicate recompute tasks from stacking up if multiple dashboard reads land while a computation is already in flight — a standard cache-stampede prevention pattern.

4. Key Architectural Decisions

Why Celery + Beat instead of FastAPI BackgroundTasks?

FastAPI's built-in background tasks share the event loop with the API server. For 19K+ row processing, this would degrade API response times. Celery isolates compute entirely — the API issues a 202 Accepted instantly and the worker handles the rest independently.

Why -P solo for the worker?

Celery's default multiprocessing prefork model conflicts with async code. Solo mode runs a single-threaded execution model inside the worker, avoiding async context conflicts when writing through the async SQLAlchemy session used elsewhere in the codebase.

Why event-driven cache recomputation instead of a fixed timer?

The original design used Celery Beat to blindly recompute analytics on a fixed interval regardless of whether new data had arrived. This wasted compute and Redis requests during idle periods, and was the root cause of Bug 7's invalidation storm. Recomputation is now triggered by the event that actually changes underlying data — the end of a batch insert — rather than by a clock.

Why dual-role Redis?

Using Upstash Redis as both the Celery message broker and the analytics cache eliminates a second managed service and keeps the architecture's moving parts to a minimum.

5. Honest Production Status

Component	Status
Sentiment-Web (FastAPI on Render)	LIVE — kept warm via scheduled health-check ping
Sentiment-Worker	NOT continuously running — started on demand, locally
Sentiment-Beat	NOT continuously running — recompute triggered manually
Upstash Redis	LIVE — serving last cached snapshot
Neon PostgreSQL	LIVE — 19,000+ rows persisted permanently
Frontend (Lovable)	LIVE — displays cached snapshot, not a real-time feed

This status table is intentionally precise rather than impressive. The distinction between “always live” (API, dashboard, database, cache) and “run on demand” (worker, Beat) is a deliberate, explainable engineering tradeoff driven by free-tier hosting constraints — not an oversight.

Built by Lakshya Singh Kushwah — github.com/Laksh-tech